Generative Priors for Unsupervised Word Discovery and Decipherment

CS787: Generative Artificial Intelligence — final report

Abstract

We study whether an unknown text can be deciphered when even its word boundaries are missing, and whether neural generative models make this easier than the classical Bayesian and n-gram approaches. English text from TinyStories is enciphered by replacing each word type with a random variable-length symbol code and deleting word boundaries. The system sees only the unsegmented cipher and a separate, non-parallel English corpus. We build a two-stage pipeline: unsupervised word segmentation (Dirichlet-process and hierarchical-Dirichlet-process samplers, an entropy heuristic, and a neural character language model) followed by key recovery (frequency matching, iterative EM with an n-gram LM, EM with word-pair states, embedding alignment, annealed key search with n-gram and neural scorers, and an LLM-based corrector). With oracle boundaries, EM already recovers 89–99% of tokens, and an LLM corrector adds one to two points on top. With learned boundaries, accuracy collapses to 8–55%. **Segmentation quality, not the strength of the English prior, is the dominant limit on end-to-end recovery.**

Code, data and resources

- **Code and results (GitHub):** https://github.com/aayushs22/CS787-Generative-Artificial-Intelligence-Project
- **Dataset (TinyStories):** https://huggingface.co/datasets/roneneldan/TinyStories
- **Detailed result tables:** `RESULTS.md` in the repository; **project proposal:** `CS787_Project_Proposal.md`.
- Raw per-run outputs (cipher text, keys, decoded text) are generated by the configs in the repository and are not stored in it.

1. Introduction

Machine translation normally needs parallel data or a dictionary. For low-resource or undeciphered languages neither exists, so one must lean on a generative model of the target language. Goldwater, Griffiths and Johnson (2006) discover word boundaries in continuous text with Bayesian models; Ravi and Knight (2011) recover a word-substitution key using EM guided by an English n-gram LM. These problems are normally studied apart: decipherment assumes segmented input and segmentation stops at boundaries. Both are driven by the same ingredient — a model of plausible language — so we ask whether modern neural models can serve as stronger priors for both stages, and we study the full path from unsegmented symbols to English.

Research questions.

- RQ1: Does a character-level neural LM, trained only on the cipher, segment as well as the Bayesian DP/HDP models?

- RQ2: How much decipherment accuracy is lost when predicted boundaries replace oracle boundaries?
- RQ3: Does a neural prior help decipherment, compared with n-gram priors?
- RQ4: How do segmentation and decipherment change with cipher corpus size?

2. Problem setup and data

Cipher. Each distinct English word type receives a unique code of random length 1–5 over a 20-symbol alphabet; codes need not be prefix-free, so segmentation is genuinely ambiguous. Sentence boundaries are given. A length-correlated encoding is used as an ablation.

Data. TinyStories, lowercased with punctuation removed, sentences of 3–12 words. The cleaned sentences are shuffled into two disjoint pools: one is enciphered, the other is the English LM corpus (20,000 sentences for N=500, 60,000 for N=2,000), so plaintext and cipher are never parallel. Both pools keep only sentences whose words are within the top-N types of the English pool. We use N=500 and N=2,000 and cipher sizes of 1,000, 5,000 and 20,000 sentences.

What the system sees. Only the unsegmented cipher and the English corpus. The gold boundaries and key are used only for evaluation.

3. Methods

3.1 Segmentation

- **Oracle / perturbed.** Gold boundaries, or gold boundaries with 10% of words randomly merged or split.
- **Entropy.** Place a boundary where branching entropy over a short symbol context is high.
- **DP unigram and HDP bigram** (Goldwater et al.). Gibbs sampling over boundary positions with simulated annealing, using the corrected conditional equations of Goldwater et al. (2009). 100 iterations, annealed in 10 steps.
- **Neural character LM.** A small decoder-only transformer (3 layers, d=96) trained on the cipher alone; boundaries go where next-symbol predictive entropy is high.

3.2 Decipherment

- **Frequency baseline.** Match cipher types to English types by frequency rank.
- **Iterative EM** (Ravi and Knight, 2011, §2.1). An HMM whose transitions are a fixed English bigram LM (Witten–Bell) and whose emissions are the unknown channel P(c|e). The channel is grown over the most frequent types, confident mappings are fixed between stages, and the final key comes from beam-restricted Viterbi decoding.
- **Key search.** Starting from the EM key, simulated annealing proposes reassignments and swaps that keep the key one-to-one; each candidate is scored by an English LM on a fixed sample of decoded sentences. Scorers: bigram LM (`search_ngram2`), trigram LM (`search_ngram3`), and a neural LM with random proposals (`search_neural`). `search_llm` draws proposals from the neural LM's next-word distribution instead.
- **LLM refinement** (`llm_refine`). A single from-scratch decoder-only LM is pretrained on the English corpus. It is then fine-tuned (LoRA) as a *corrector* that maps a noisily decoded sentence

to a clean one, where noise swaps a random share of word types with words of similar frequency rank. At test time the corrector rewrites every decoded sentence; each cipher unit takes the word it was most often corrected to (one-to-one, greedy by votes), and the new key is kept only if the LM score improves. The LM never sees cipher text or the key.

- **Word-pair keys** (`em_pairs`, proposal §4.3). Segmentation can fuse two frequent words into one unit. This variant of EM adds the 50 most frequent English bigrams as extra HMM states; a pair state emits a single cipher unit but decodes to two words, with transition factors P(first word | previous word) and the internal P(second | first). A pair-mapped unit appears in the key as `"w1 w2"`, and decoded sentences are flattened to words. With 0 pair states it reduces to plain EM.
- **Embedding alignment** (`embed_align`). Word embeddings (PPMI + SVD, 30 dimensions) are trained independently on the segmented cipher and on English, and aligned without any dictionary: an initial dictionary comes from sorted self-similarity profiles (Artetxe et al., 2018), then iterative Procrustes refinement with CSLS retrieval (Conneau et al., 2018), and a final one-to-one assignment gives the key. Unlike the proposal, initialisation is not adversarial.

3.3 Metrics

Segmentation: boundary, word-token and lexicon precision/recall/F (Goldwater et al.). Decipherment: token accuracy after an edit-distance alignment between decoded and reference sentences (well defined even when the number of units differs), sentence accuracy, and vocabulary recovery (the share of word types whose true code is a predicted unit mapped to the right word).

4. Results

All numbers are token accuracy unless stated. N=500 unless stated. Every cell with a single seed is one run; only the 1,000-sentence setting was repeated with three seeds (0, 1, 2). Full tables are in RESULTS.md.

4.1 Segmentation (RQ1)

Boundary / token / lexicon F at 1,000 sentences, mean of three seeds:

segmenter	boundary F	token F	lexicon F
dp_unigram	0.830	0.672	0.664
hdp_bigram	0.779	0.543	0.526
charlm	0.570	0.311	0.254

Boundary F as the cipher grows (seed 0):

segmenter	1k	5k	20k
dp_unigram	0.811	0.782	0.700
hdp_bigram	0.785	0.764	0.692
charlm	0.598	0.720	0.713

At 1,000 sentences the character LM is clearly the weakest segmenter in all three seeds. Its quality improves with data, while the Gibbs samplers get *worse* (100 iterations are likely too few for large

corpora; we did not check convergence). The neural segmenter has a very low lexicon F at 20k (0.12), i.e. it proposes many spurious unit types. HDP did not clearly outperform DP, so we did not reproduce the under-segmentation gap that Goldwater et al. report for the unigram model.

4.2 Cost of segmentation errors (RQ2)

EM token accuracy at 1,000 sentences:

segmentation	seed 0	seed 1	seed 2
oracle	0.884	0.889	0.887
oracle + 10% perturbation	0.718	–	–
dp_unigram	0.394	0.553	0.397
hdp_bigram	0.125	0.403	0.235
charlm	0.166	0.111	0.088
entropy	0.398	–	–

EM token accuracy across corpus sizes and vocabularies (seed 0):

setting	oracle	dp_unigram	hdp_bigram	charlm
N=500, 1k	0.884	0.394	0.125	0.166
N=500, 5k	0.968	0.200	0.325	0.341
N=500, 20k	0.985	0.083	0.166	0.417
N=2000, 5k	0.836	0.423	0.320	0.227

This is the largest effect in the study. Randomly corrupting only 10% of the words already costs about 17 points. Learned segmentations lose 30–90 points relative to oracle boundaries, and — unlike oracle accuracy — their downstream accuracy does not improve reliably with more data. The learned-segmenter results also vary substantially across seeds (e.g. HDP: 0.125, 0.403, 0.235), so individual cells should be read as indicative.

4.3 Neural versus n-gram priors (RQ3)

Oracle boundaries, 1,000 sentences, mean of three seeds:

method	token acc.
frequency baseline	0.238
EM (bigram)	0.887
key search, bigram LM	0.891
key search, neural LM	0.894
LLM refinement	0.903

Larger settings, oracle boundaries (EM → LLM refinement): 5k 0.968 → 0.980; 20k 0.985 → 0.994; N=2000 (5k) 0.836 → 0.850.

With oracle boundaries the LLM corrector beats its EM starting point in every setting and in all three 1k seeds (by 1.2–2.0 points); the three key-search variants are within noise of each other and of EM. In the 1k oracle runs EM and LLM refinement have the same vocabulary recovery (0.643), and refinement's gain is concentrated in tokens rather than types (sentence accuracy 0.493 → 0.510). Under learned segmentation no method is consistently ahead. Because EM is already close to ceiling with a clean segmentation, there is little room for a stronger prior to show a large effect; we did not test whether the neural advantage grows when the cipher corpus is smaller than 1,000 sentences.

4.4 Data size and vocabulary (RQ4)

With oracle boundaries, EM accuracy rises from 0.89 (1k) to 0.97 (5k) to 0.985 (20k); a larger vocabulary is harder (N=2,000 with 5k sentences: 0.84 against 0.97 for N=500). Segmentation does not benefit comparably (§4.1–4.2), so more cipher data helps decipherment only when the boundaries are given.

4.5 Length-correlated codes

With length-correlated codes the oracle result is unchanged (EM 0.884), and segmentation F moves by only a few points (char-LM boundary F 0.60 → 0.57, DP 0.81 → 0.77, HDP 0.79 → 0.76). Downstream accuracy under learned segmentation moves in both directions by amounts as large as seed-to-seed variation (DP+EM 0.39 → 0.16, HDP+EM 0.13 → 0.38), so we find no clear evidence either way that code length leaks into the segmenter.

4.6 Word-pair keys and embedding alignment

Token accuracy; runs reuse the same data and seeds as above (the 20k run omits `hdp_bigram`):

setting	segmentation	em	em_pairs	embed_align
1k, seeds 0 / 1 / 2	oracle	.884 / .889 / .887	.717 / .762 / .770	.059 / .067 / .044
1k, seeds 0 / 1 / 2	dp_unigram	.394 / .553 / .397	.550 / .690 / .601	.018 / .031 / .023
5k	oracle	.968	.888	.109
5k	dp_unigram	.200	.256	.024
20k	oracle	.985	.904	.600
20k	dp_unigram	.083	.070	.025

Word-pair keys help when units are fused and hurt otherwise. With DP unigram segmentation at 1k, `em_pairs` beats `em` in all three seeds, by 14–20 points. With oracle boundaries it costs 11–17 points, because the extra pair states add spurious alternatives. At 5k the gain with DP is smaller (+5.6) and at 20k it disappears (−1.3). With HDP and character-LM boundaries the effect is mixed (HDP at 1k: +4.5, −19.5, −8.7 for seeds 0/1/2). The number of pair states was not tuned, and the method has no way to decide per unit whether a pair is warranted.

Embedding alignment is weak at this scale. Even with oracle boundaries it is below the frequency baseline (0.23–0.29) at 1k and 5k, reaches 0.60 at 20k (above the frequency baseline of 0.39 but far below EM's 0.985), and is near zero under learned segmentation. It needs far more cipher data than EM. Its settings (30 dimensions, 300 types, 100 initialisation types) were chosen by trying four combinations on the 5k and 20k oracle data, i.e. on evaluation data, so the 20k figure is optimistic.

5. Discussion

The central hypothesis was that stronger priors improve both stages, but that segmentation errors remain the main bottleneck. The evidence supports the second half firmly and the first half only weakly:

- **Segmentation is the bottleneck.** The gap between oracle and learned boundaries (tens of points) dwarfs every difference between decipherment priors (about one to two points). It also persists as data grows.
- **A neural prior helps, a little.** The LLM corrector is the best decipherment method with clean boundaries, but its margin over a bigram EM is small and we measured it on a task where EM is already strong.
- **Handling segmentation errors in the key helps only in the matching regime.** Word-pair keys recover part of the loss when a segmenter fuses words (DP unigram at small scale) and cost accuracy when boundaries are already right.
- **A neural prior for segmentation did not help at small scale.** The character LM was the worst segmenter at 1k sentences and only became competitive at 5k–20k, where it produced the best end-to-end accuracy among learned segmenters (0.34 at 5k and 0.42 at 20k with EM) despite poor lexicon F.

6. Limitations

- **Single seeds.** Only the 1k setting has three seeds; all other results are single runs, and the learned segmenters are high-variance.
- **No tuning.** Samplers, the character LM, the LLM and EM use fixed placeholder hyperparameters. The Gibbs samplers use 100 iterations at all corpus sizes and are probably under-converged at 20k sentences.
- **Controlled upper bound.** The key is a one-to-one relabelling of English, so these numbers bound what is achievable on real languages, which add non-deterministic translation, reordering and morphology.
- **Key search start.** Key search and refinement start from the EM key, so they can only improve on what EM finds.
- **Scale.** The LM and corrector are small (about a million parameters or fewer) and trained for a few thousand steps on a laptop CPU.
- **Embedding alignment** uses a non-adversarial initialisation and settings picked on evaluation data (§4.6).
- **Not tested.** The effect of scarce cipher data (below 1,000 sentences) on the neural advantage.

7. Implementation and reproducibility

All code is in `decipher/`; the pipeline is `python -m decipher.run --config configs/<name>.yaml`, and results tables are produced by `python -m decipher.report runs/<name>`. The configs used here are `configs/n500_c1000.yaml`, `n500_c1000_s1.yaml`, `n500_c1000_s2.yaml`, `n500_c5000.yaml`, `n500_c20000.yaml`, `n2000_c5000.yaml` and `n500_c1000_corr.yaml`; the word-pair and embedding-alignment runs use `ext_c1000_s0.yaml`, `ext_c1000_s1.yaml`, `ext_c1000_s2.yaml`, `ext_c5000.yaml` and `ext_c20000.yaml`. Embedding alignment needs `scipy`. `configs/smoke.yaml` is a toy configuration for quick checks; its values are placeholders. The test suite (`pytest tests`) covers

forward–backward against brute force, Gibbs count consistency, the metrics, word-pair EM, embedding alignment, the LM and the fine-tuning techniques.

Two points about the code base. The decoder models and fine-tuning techniques (`decipher/models/`, `decipher/techniques/`) come from the team's earlier capstone code, with the small fixes marked `# FIX (Claude)`. And while running the 20k-sentence experiment we found and fixed a numerical bug in the HDP sampler: accumulated floating-point error left an expected table count slightly negative, which made rare-unit probabilities negative and, under annealing, complex-valued. The count is now clamped at zero; the 1k and 5k runs completed without hitting the bug and were not rerun.

References

- Goldwater, S., Griffiths, T. L., & Johnson, M. (2006). Contextual Dependencies in Unsupervised Word Segmentation. *ACL 2006*. https://aclanthology.org/P06-1085/
- Goldwater, S., Griffiths, T. L., & Johnson, M. (2009). A Bayesian framework for word segmentation: Exploring the effects of context. *Cognition* 112(1).
- Ravi, S., & Knight, K. (2011). Deciphering Foreign Language. *ACL-HLT 2011*. https://aclanthology.org/P11-1002/
- Artetxe, M., Labaka, G., & Agirre, E. (2018). A robust self-learning method for fully unsupervised cross-lingual mappings of word embeddings. *ACL 2018*.
- Conneau, A., Lample, G., Ranzato, M., Denoyer, L., & Jégou, H. (2018). Word Translation Without Parallel Data. *ICLR 2018*. https://arxiv.org/abs/1710.04087
- Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759. https://arxiv.org/abs/2305.07759